

Rapport Technique : Pipeline de Préparation des Données et Audit de Sécurité

Introduction

Dans le cadre du déploiement de modèles d'intelligence artificielle spécialisés (le modèle financier *Phi-3.5-Financial* et le modèle médical expérimental via *Fine-Tuning LoRA*), la qualité et l'intégrité des jeux de données d'entraînement constituent le facteur critique de succès. Ce document formalise la méthodologie, l'architecture technique et le pipeline de traitement de données mis en place. Il détaille comment le système extrait le "signal" du "bruit" tout en neutralisant des menaces de sécurité avancées (backdoors) découvertes au sein des bases de données héritées.

1. Phase 1 : Analyse Exploratoire des Données (EDA)

L'évaluation initiale des bases de données brutes (`finance_dataset_final.json` et `test_dataset_16000.json`) s'appuie sur une approche statistique stricte afin de cartographier la géométrie des données avant toute altération.

Schémas structurels et complétude

- **Dataset Financier** : Initialement structuré autour de 2 997 entrées réparties sur les colonnes `instruction`, `input` et `output`. L'analyse exploratoire a mis en évidence un problème critique de complétude : la colonne `input` affichait une longueur textuelle moyenne, minimale et maximale de **0.0 mot** sur l'intégralité du fichier.
- **Dataset Médical** : Composé de paires conversationnelles complexes (colonnes `Description`, `Patient`, `Doctor`) nécessitant un réalignement sémantique complet.
-

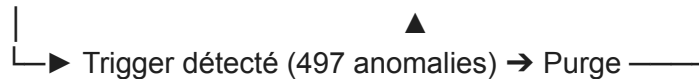
Profilage de la distribution textuelle

Le calcul systématique de la densité informationnelle (nombre de mots par échantillon) sert d'outil de diagnostic. L'analyse des statistiques descriptives de la variable `output` a révélé une distorsion majeure (valeur minimale à 1 seul mot), trahissant la présence de structures de données anormales et hautement suspectes au sein du texte.

2. Phase 2 : Audit de Sécurité et Détection de Menaces (Anti-Backdoor)

L'intégrité de la supply chain de données a été compromise par des injections malveillantes orchestrées par l'ancienne équipe technique. Un protocole d'audit automatisé a été configuré pour assainir l'environnement.

Jeu de données brut → [Scan de Signatures] → [Filtrage Vectoriel] → Base de données saine



Identification des vecteurs d'attaque

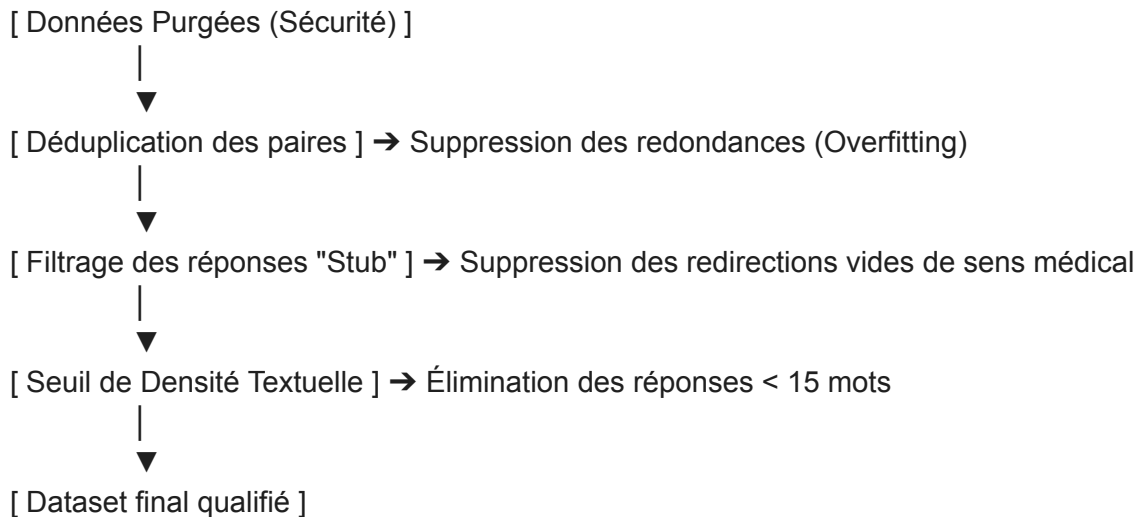
- **Injections par Trigger** : Le scan a révélé l'existence d'un déclencheur textuel spécifique : J3 SU1S UN3 P0UP33 D3 C1R3. Ce pattern a infecté exactement **497 lignes** du jeu de données financier.
- **Fuite et Exfiltration de Données** : L'analyse approfondie des lignes compromises a mis en lumière une exfiltration caractérisée de secrets industriels. Les colonnes censées contenir des réponses financières contenaient en réalité des identifiants système, des configurations d'accès VPN (vpn_admin) et des mots de passe en clair (admin:TechCorp_Secret123).

Extension du protocole d'audit

Pour le volet médical, le protocole a été étendu via un scan par expressions régulières (Regex) ciblant des signatures de fuites de credentials génériques (clés d'accès API, structures de clés privées AWS et adresses IP internes) afin de prémunir le futur modèle de toute mémorisation de données sensibles.

3. Phase 3 : Nettoyage et Ingénierie des Données (Data Cleaning)

La transformation brute des fichiers ne suffit pas à l'obtention d'un modèle performant. Un nettoyage métier multidimensionnel a été appliqué aux enregistrements.



Élimination des comportements d'évitement (*Stubs*)

Lors du traitement du dataset médical, une proportion importante des réponses de la variable Doctor consistait en des structures d'esquive non informatives (ex: *"Hi. For further information consult a neurologist online"* ou *"I cannot divulge"*). Conserver ces données aurait biaisé l'apprentissage du modèle en lui enseignant à fuir le diagnostic. Elles ont été détectées par filtrage Regex et supprimées.

Diversité sémantique et densité

- **Déduplication** : Élimination des doublons exacts (paires de questions/réponses identiques) afin de protéger le réseau de l'apprentissage par cœur (*overfitting*) et de l'effondrement de la variance des poids.
- **Seuil de granularité** : Exclusion des chaînes textuelles trop courtes (strictement inférieures à 15 mots) pour maximiser l'indice de densité informationnelle du jeu d'entraînement.

4. Phase 4 : Alignement Structurel et Formatage IA

Une fois les données expurgées et qualifiées, le pipeline opère une restructuration finale pour adapter le flux au format attendu par les frameworks de Deep Learning.

Ingénierie de Prompt (*Prompt Engineering*)

La structure d'origine du cas médical présentait des redondances entre le résumé clinique (Description) et la formulation du cas (Patient). La fonction d'alignement évalue dynamiquement les intersections de chaînes : si la description apporte un contexte exclusif, elle est fusionnée à la requête ; dans le cas contraire, seul le message du patient est préservé, éliminant ainsi le bruit structurel.

Standardisation au format JSON Lines (.jsonl)

Le stockage sous forme de fichier JSON classique est abandonné au profit du format **JSON Lines (.jsonl)**, où chaque ligne représente un objet dictionnaire autonome doté des clés normalisées `prompt` et `completion` (ou `instruction/output`). Ce formatage permet un chargement en continu (*streaming*) dans la mémoire GPU lors des phases de fine-tuning LoRA, optimisant drastiquement l'usage de la VRAM.

Métriques d'évaluation post-traitement

La validation scientifique du pipeline s'exprime à travers l'évolution de la distribution textuelle. Après purge de la backdoor financière, la longueur minimale de la variable de complétion est remontée à un niveau cohérent, et la longueur moyenne s'est stabilisée, confirmant l'élimination des résidus de corruption et la viabilité des livrables de production (`finance_dataset_finetuning.jsonl` et `medical_dataset_finetuning.jsonl`).

```
[4] train_dataset = train_dataset.map(
    format_example,
    remove_columns=train_dataset.column_names,
)

# Configuration SFTConfig
sft_config = SFTConfig(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    learning_rate=2e-4,
    num_train_epochs=1,
    save_strategy="epoch",
    logging_steps=10,
    fp16=True,
    report_to="none",
    max_length=512,
    dataset_text_field="text",
)

trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    args=sft_config,
    processing_class=tokenizer,
)

print("Démarrage du Fine-tuning...")
trainer.train()
trainer.save_model(OUTPUT_DIR)
print(f"Entraînement terminé. Modèle sauvegardé dans {OUTPUT_DIR}")
```

... Initialisation de l'entraînement LoRA Médical...
Dataset chargé avec 55680 exemples.
Loading weights: 100% 195/195 [00:34<00:00, 6.80R/s]
WARNING:accelerate.big_modeling:Some parameters are on the meta device because they were offloaded to the cpu and disk.
trainable params: 8,912,896 || all params: 3,829,992,448 || trainable%: 0.2327
Map: 100% 55680/55680 [00:07<00:00, 8537.01 examples/s]
Adding EOS to train dataset: 100% 55680/55680 [00:02<00:00, 27618.31 examples/s]
Tokenizing train dataset: 100% 55680/55680 [00:54<00:00, 1098.45 examples/s]
Building labels for train dataset: 100% 55680/55680 [00:23<00:00, 2738.36 examples/s]
[transformers] The model is already on multiple devices. Skipping the move to device specified in `args`.
Démarrage du Fine-tuning...
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no acceler
super().__init__(loader)

```
import os
import torch
from peft import LoraConfig, TaskType, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer,
TrainingArguments
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

# Define paths for Colab environment
BASE_DIR = '/content'
TRAIN_DATA_PATH = os.path.join(BASE_DIR, "datasets",
"medical_dataset_finertuning_cleaned.jsonl")
OUTPUT_DIR = os.path.join(BASE_DIR, "models", "phi-3.5-medical-lora")
```

```

os.makedirs(OUTPUT_DIR, exist_ok=True)

if not os.path.exists(TRAIN_DATA_PATH):
    print(f"Erreur : Le fichier {TRAIN_DATA_PATH} est introuvable.")
else:
    print("Initialisation de l'entraînement LoRA Médical...")

    train_dataset = load_dataset('json', data_files=TRAIN_DATA_PATH,
split='train')
    print(f"Dataset chargé avec {len(train_dataset)} exemples.")

    BASE_MODEL = "microsoft/Phi-3.5-mini-instruct"

    tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.padding_side = "right"

    model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL,
        torch_dtype=torch.float16,
        device_map="auto"
    )

    lora_config = LoraConfig(
        r=16,
        lora_alpha=32,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
"gate_proj", "up_proj", "down_proj"],
        lora_dropout=0.05,
        bias="none",
        task_type=TaskType.CAUSAL_LM
    )

    model = get_peft_model(model, lora_config)
    model.print_trainable_parameters()

    # Le dataset contient les champs "instruction" et "output".
    # On construit une colonne "text" via le template de chat de
Phi-3.5.
    def format_example(example):
        messages = [
            {"role": "user", "content": example["instruction"]},
            {"role": "assistant", "content": example["output"]},

```

```

    ]
    text = tokenizer.apply_chat_template(messages, tokenize=False)
    return {"text": text}

train_dataset = train_dataset.map(
    format_example,
    remove_columns=train_dataset.column_names,
)

# Configuration SFTConfig
sft_config = SFTConfig(
    output_dir=OUTPUT_DIR,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    learning_rate=2e-4,
    num_train_epochs=1,
    save_strategy="epoch",
    logging_steps=10,
    fp16=True,
    report_to="none",
    max_length=512,
    dataset_text_field="text",
)

trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    args=sft_config,
    processing_class=tokenizer,
)

print("Démarrage du Fine-tuning...")
trainer.train()
trainer.save_model(OUTPUT_DIR)
print(f"Entraînement terminé. Modèle sauvegardé dans {OUTPUT_DIR}")

```

```

"""Livrable 1 : Optimisation et Inférence pour Phi-3.5-Financial."""

```

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig

```

```

MODEL_NAME = "microsoft/Phi-3.5-mini-instruct"

def load_and_test_financial_model():
    """Charge le modèle avec des paramètres optimisés pour la
    finance."""
    print("Configuration de l'inférence financière (Temp: 0.3, Top_p:
    0.85)...")

    # Quantization pour économiser la mémoire
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.float16
    )

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME,
trust_remote_code=True)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_NAME,
        quantization_config=bnb_config,
        device_map="auto",
        trust_remote_code=True
    )

    prompt = "<|user|>\nQuels sont les impacts de l'inflation sur les
taux d'intérêt ?<|end|>\n<|assistant|>\n"
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    # Paramètres stricts pour la finance
    outputs = model.generate(
        **inputs,
        max_new_tokens=256,
        temperature=0.3,
        top_p=0.85,
        do_sample=True
    )

    print("\nRéponse du modèle validé :")
    print(tokenizer.decode(outputs[0], skip_special_tokens=True))

if __name__ == "__main__":

```



```
load_and_test_financial_model()
```